

Integrity Solve AML/CTF Compliance Portal

Full Code Implementation Plan

Multi-tenant SaaS architecture, modules, data model, integrations, workflows, milestones, and acceptance criteria

Prepared for project discussion and staged MVP delivery

Document Purpose	Important Note
This plan translates the provided scope, wireframes, permissions matrix, workflow/state matrix, API contract, backlog, and technical architecture into a practical development roadmap.	The developer can implement the platform and workflow logic, but AML/CTF legal content, final risk rules, provider choices, and document wording should be validated by the client or a qualified compliance expert.

1. Executive Summary

The project should be treated as a staged multi-tenant compliance SaaS platform, not a simple form website. The build has two operating streams:

- Build / Update My Program: each business client implements and maintains its own AML/CTF program through guided workflows and generated outputs.
- Onboard and Manage Clients: the business uses guided due diligence workflows, paid checks, escalations, reviews, and monitoring to manage its own customers.

The safest approach is a modular architecture with workspace isolation, role-based access control, workflow-driven forms, document generation, provider-agnostic API adapters, usage-based billing, task automation, and audit trails from day one.

Core Goal	Build Direction
Secure portal foundation	Authentication, workspace creation, RBAC, route guards, first-login restricted dashboard.
Guided AML implementation	Six-step setup wizard, autosave, structured form data, progress tracking, generated risk/policy/procedure outputs.
Client onboarding engine	Customer files, CDD/ECDD, beneficial ownership, source of funds/wealth, escalations, periodic review.
Paid checks and billing	Identity/KYB/AML provider adapters, check requests, webhook handling, usage events, Stripe billing.
Compliance-grade records	Immutable audit logs, versioned generated documents, evidence storage, role-based approvals.

2. Recommended Technical Stack

Layer	Recommended Choice	Reason
Frontend	Next.js + React + TypeScript	Strong dashboard UX, protected routes, server/client flexibility, fast development.
UI	Tailwind CSS + shadcn/ui	Clean enterprise UI, reusable components, fast form/dashboard development.
Backend	Node.js TypeScript - NestJS preferred, Express acceptable for lean MVP	Modular services, DTOs, auth guards, provider adapters, maintainable APIs.
Database	PostgreSQL + Prisma ORM	Relational model fits workspaces, forms, roles, billing, tasks, audit logs, provider results.
Auth	Clerk/Auth0 or custom JWT	Multi-user workspaces, invitations, MFA option, role handling.
Storage	AWS S3 / Cloudflare R2 / Supabase Storage	Generated docs, uploaded evidence, raw provider reports, signed URLs.
Queue/Jobs	BullMQ + Redis or managed queue	Document generation, provider polling, webhook retry, reminders, billing sync.
Billing	Stripe	Subscriptions, payment methods, invoices, metered/usage events.
Monitoring	Sentry + structured logs	Track API failures, job failures, billing mismatches, provider issues.
Deployment	Vercel frontend + Railway/Fly/AWS backend + managed Postgres	Fast MVP deployment while keeping backend and jobs under control.

3. High-Level Architecture

```
Public Site
-> Registration + Identity Verification
-> Business Registration + Workspace Creation
-> Restricted First-Login Dashboard
-> Implementation Wizard (6 setup forms)
-> Document Generation (Risk Assessment, Policy, Procedures)
-> Full Portal
```

- > Manage My Program
- > Onboard and Manage Clients
- > Billing and Usage
- > Settings / Support

Backend Services

Auth/RBAC Service
 Workspace Service
 Program Forms Service
 Workflow State Service
 Document Generation Service
 Customer File Service
 Provider Integration Service
 Billing/Usage Service
 Task/Notification Service
 Audit Log Service

3.1 Architecture Rules

- Every business client is a workspace/tenant. All records must be scoped by workspace_id.
- Never rely only on frontend hiding for permissions. All sensitive actions must be checked on the backend.
- Provider integrations must use adapters behind common capability contracts, not hard-coded UI workflows.
- Workflow state transitions should be explicit and logged, not inferred only from form completion.
- Documents are generated outputs from structured data. They should be versioned and reproducible.
- Paid checks should create usage events and audit logs, with idempotency to avoid duplicate charges.

4. Repository and Code Structure

A monorepo is recommended for speed and shared types. The structure below keeps frontend, backend, shared contracts, and infrastructure clear.

```
/apps
  /web                                # Next.js frontend
  /app
    /(public)
    /(auth)
    /app/dashboard-onboarding
    /app/setup/[step]
    /app/dashboard
    /app/program
    /app/clients
    /app/escalations
    /app/reviews
    /app/billing
    /app/settings
  /components
    /forms
    /workflow
    /dashboard
    /tables
    /documents
    /rbac
  /lib
    api-client.ts
    auth.ts
    route-guards.ts
```

form-schemas.ts

```
/api                                # NestJS/Node backend
/src
  /modules
    auth
    users
    workspaces
    memberships
    rbac
    program-forms
    program-versions
    documents
    customers
    customer-files
    screening
    providers
    billing
    tasks
    notifications
    audit
    webhooks
  /common
    guards
    decorators
    interceptors
    filters
    dto
    enums
    utils
  main.ts
```

```
/packages
  /shared                          # Shared DTOs, enums, Zod schemas, API contracts
  /config                          # ESLint, TS config, environment config
  /database                        # Prisma schema, migrations, seed scripts
```

```
/infra
  docker-compose.yml              # local Postgres/Redis
  deployment.md
  env.example
```

5. Core Data Model

The database should be designed around structured entities, not just uploaded documents. Documents are outputs generated from structured forms and workflow events.

Entity / Table	Purpose	Key Fields
users	Login identity and personal profile	id, email, mobile, identity_status, is_active, created_at
workspaces	Business tenant/account	id, legal_name, trading_name, abn, acn, registry_status, implementation_status, billing_status
workspace_memberships	User role inside a workspace	id, user_id, workspace_id, role, status, invited_by, joined_at

business_profiles	Versionable business profile details	workspace_id, industry_pathway, address, authority_declaration, data_json
program_forms	Implementation and maintenance forms	workspace_id, step_code, status, data_json, completed_at, submitted_by
program_versions	Generated AML program snapshot	workspace_id, version_number, risk_snapshot_json, effective_date, generated_by
program_documents	Generated risk/policy/procedure docs	program_version_id, doc_type, file_path, format, checksum, generated_at
customers	Customer master record	workspace_id, customer_type, legal_name, status, risk_rating
customer_files	CDD relationship/matter file	workspace_id, customer_id, file_reference, state, next_review_due
customer_forms	CDD/ECDD/BO/SOF/escalation/review forms	customer_file_id, form_type, status, data_json, decision
beneficial_owners	Owners/controllers linked to entity customers	customer_file_id, name, ownership_percent, country, screening_status
check_requests	Internal paid/external check request	workspace_id, subject_type, subject_id, provider_capability, status, request_id
check_results	Normalized provider result	check_request_id, provider_request_id, decision, result_json, raw_payload_ref
usage_events	Metered billing unit	workspace_id, check_request_id, event_type, quantity, amount, invoice_status
tasks	Human/action workflow tasks	workspace_id, customer_file_id, source_type, status, assignee_id, due_at
audit_logs	Immutable audit record	workspace_id, actor_user_id, action, entity_type, entity_id, old_value, new_value, reason
webhook_events	Raw provider webhook events	provider_name, provider_request_id, event_type, signature_valid, processed_at
provider_connections	Provider config/credentials pointer	provider_name, capability, environment, secret_ref, status

6. Backend Module Plan

Module	Responsibilities	Important Endpoints / Jobs
Auth / Users	Account creation, identity status, email/mobile verification hooks, session handling.	POST /auth/register, GET /me, PATCH /users/:id/status
Workspaces	Business registration, ABN/ACN details, workspace state, setup progress.	POST /workspaces, GET /workspaces/:id, PATCH /workspaces/:id
Memberships / RBAC	Invites, role assignment, permission checks, support access.	POST /memberships/invite, PATCH /memberships/:id/role
Program Forms	Six-step implementation wizard, autosave, validation, form versions.	GET /program/forms, PUT /program/forms/:step, POST /program/forms/:step/submit
Workflow State	Workspace state transitions, client file states, lock/unlock rules.	POST /workflow/transition, GET /workflow/state
Documents	Template mapping, DOCX/PDF generation, version storage, download links.	POST /documents/generate, GET /documents/:id/download
Customers / Files	Customer records, CDD files, onboarding workflows.	POST /customers, POST /customer-files, GET /customer-files/:id
CDD Forms	Initial CDD, ECDD, BO, SOF/SOW, escalation, unusual activity, review forms.	PUT /customer-files/:id/forms/:type, POST /customer-files/:id/forms/:type/submit
Providers	Adapter interface, request creation, polling, normalized results.	POST /checks, GET /checks/:id, background provider jobs
Webhooks	Provider webhook validation, idempotency, result update, side effects.	POST /webhooks/:provider

Billing	Stripe customers/subscriptions, usage events, invoices, payment method state.	POST /billing/setup, GET /billing/usage, webhook stripe events
Tasks / Notifications	Task generation, assignment, due dates, email/in-app notifications.	GET /tasks, PATCH /tasks/:id, scheduled review jobs
Audit	Central event logger for security/compliance actions.	Internal audit service + GET /audit with RBAC

7. Frontend Module and Route Plan

Route / Screen	Purpose	Core Components
/	Public marketing and CTA entry	Hero, value cards, pricing CTA, login/register buttons
/register	Create user account	RegisterForm, TermsCheckbox, EmailVerificationStatus
/verify-identity	Identity verification step	ProviderSessionLauncher, VerificationStatusCard, RetryAction
/register-business	Business details + ABN/ACN lookup	BusinessForm, RegistryLookupCard, AuthorityDeclaration
/app/dashboard-onboarding	Restricted first-login dashboard	SetupProgressCard, LockedFeaturePreview, BillingStatusCard, ContinueSetupCTA
/app/setup/[step]	Six-step AML implementation wizard	WizardLayout, StepNav, DynamicFormRenderer, AutosaveStatus, ContextHelpPanel
/app/dashboard	Full unlocked portal dashboard	ProgramHealth, RiskSummary, DocumentsPanel, TasksWidget, ClientActivity, UsageSnapshot
/app/program/*	Manage My Program	ProgramForms, TrainingRecord, ProgramReviewUpdate, DocumentVersions
/app/clients/new	New client onboarding	CustomerTypeSelector, CDDWizard, PaidCheckTrigger, RiskResultPanel
/app/clients/[id]	Customer file detail	FileTimeline, FormsTab, ChecksTab, EvidenceTab, DecisionsTab, AuditTab
/app/escalations	Escalation queue	EscalationTable, ReviewDecisionModal, AssignReviewer
/app/reviews	Periodic review queue	ReviewDueTable, ReviewWorkflowPanel
/app/billing	Subscription and usage	PaymentMethodCard, UsageLedger, InvoiceList, PaidCheckSettings
/app/settings	Users, roles, workspace settings	UserInviteForm, RoleMatrix, IntegrationSettings, NotificationsSettings

8. Workflow and State Machine Plan

The system should use explicit state machines for workspace onboarding, implementation steps, client files, escalations, periodic reviews, and provider checks.

Workflow	Core States	Notes
Workspace	registered -> identity_verified -> business_verified -> setup_in_progress -> setup_complete -> active	Controls dashboard unlock and access to client onboarding.
Billing	inactive -> setup_pending -> active -> past_due -> suspended -> cancelled	Paid checks blocked unless payment method is valid and billing rules allow it.
Implementation Step	not_started -> in_progress -> completed -> requires_revision	Autosave at each step; future steps locked until prior valid completion.
Customer File	draft -> cdd_in_progress -> pending_checks -> checks_returned -> under_review -> approved/held/declined	Core client onboarding state machine.
Escalation	open -> under_review -> action_required -> resolved/declined/closed	Reviewer decision must be audited.

Provider Check	queued -> processing -> passed/failed/partial/escalated/error	Idempotent request_id required; failures cannot be treated as clear results.
Document	draft -> generated -> approved/current -> superseded	Generated outputs must be linked to program version.

Example state transition guard:

- A workspace cannot unlock /app/clients until implementation_status = complete.
- A paid check cannot run unless billing_status = active.
- An Onboarding User can create and edit CDD, but cannot approve high-risk outcomes.
- A Reviewer/Compliance Officer can approve, hold, decline, or escalate customer file decisions.
- Every approval/rejection/override requires an audit log with user, role, timestamp, decision, and note.

9. Permission Model Plan

Start with simplified MVP roles to reduce build cost while keeping the system secure. Backend authorization should combine user role, workspace state, billing state, and action-level permissions.

MVP Role	Access Summary	Important Limitations
Workspace Admin	Workspace settings, billing, users, dashboard, program outputs.	Not automatically Compliance Officer for high-risk final decisions unless assigned.
Compliance Officer	Program management, escalations, reviews, client files, approvals.	No billing/payment access unless combined with admin role.
Onboarding User	Create client files, complete CDD, upload evidence, run allowed checks.	Cannot approve high-risk outcomes or governance changes.
Reviewer / Escalation User	Review escalations, ECDD, unusual activity, source of funds/wealth.	No billing or workspace admin by default.
Read Only / Auditor	View dashboards, documents, files and audit trail if permitted.	No edit, no approval, no paid checks.
Integrity Solve Support	Delegated support access per workspace.	Access must be explicit, revocable, and auditable.

10. Provider Integration Plan

Provider integration should be built through capability-based adapters. The UI should ask for identity_verification, aml_screening, kyb_verification, or billing_authorization, not direct vendor-specific calls.

Capability	MVP Providers / Examples	Implementation Notes
Identity verification / liveness	Facia or equivalent	Create session, handle async completion, store identity status, confidence, raw payload ref.
KYB / business registry verification	ABN/ACN/ASIC/ABR/KYB provider	Lookup business/entity, prefill fields, require user confirmation, support manual review.
AML screening	AML Watcher, Personr, My Data Boss or equivalent	Screen individuals/entities, detect PEP/sanctions/adverse media, create review tasks on matches.
Billing gateway	Stripe	Payment method, subscriptions, invoice status, usage events, billing webhooks.
Program management sync	Optional later provider	Keep out of MVP unless client explicitly confirms active sync is required.

Provider adapter contract:

```
interface ProviderAdapter {
  capability: ProviderCapability;
  createRequest(input: CheckRequestEnvelope): Promise<ProviderStartResult>;
  normalizeResponse(raw: unknown): Promise<NormalizedCheckResult>;
  verifyWebhook(signature: string, body: unknown): boolean;
  mapWebhookToRequest(body: unknown): Promise<string>; // provider_request_id -> internal request_id
}
```

Core rule:

UI action -> check_request -> provider adapter -> check_result -> usage_event -> task/audit updates

11. Billing and Usage Plan

Billing should separate subscription access from usage-based paid checks. This prevents confusion and gives the client clear reporting.

Billing Area	Implementation	Acceptance Criteria
Subscription	Create Stripe customer/subscription for each workspace.	Workspace billing status updates correctly from Stripe webhook events.
Payment Method	Store only Stripe/customer references, never raw card details.	Payment method status visible in dashboard and billing page.
Usage Events	Create usage_event from intentional paid provider actions.	Each billable check has request_id, provider, workspace, customer file, cost category.
Invoice/Usage View	Usage ledger separated from subscription charges.	Workspace admin can see checks, status, invoice state, and amounts.
Blocking Rules	Paid checks blocked if billing inactive/past_due/suspended.	User sees clear prompt instead of silent failure.

12. Document Generation Plan

Generated AML/CTF outputs should be deterministic outputs of structured form data. The user edits forms, not the generated policy directly, unless later scope explicitly adds controlled in-portal document editing.

- Document templates: My Business Risk Assessment, AML/CTF Policy, AML/CTF Procedures/SOP, Governance Summary/Completion Pack.
- Recommended output: generate DOCX first, then convert to PDF for viewing/download.
- Version every generated output with program_version_id, generated_at, generated_by, template_version, checksum, and source data snapshot.
- Store documents in S3/R2 with secure signed URLs or backend proxy download.
- Client/compliance expert must provide final legal wording, risk formulas, and template variables.

13. Security, Privacy, Audit, and Compliance Controls

Area	Implementation Requirement
Workspace isolation	Every query filters by workspace_id. Use middleware/guards to prevent cross-tenant access.
RBAC	Backend checks for role, action, workspace state, and billing state.
Provider secrets	Use secrets manager/environment secrets. Never expose keys in frontend or plain DB fields.
PII/evidence	Store sensitive files encrypted at rest; use secure links; minimize provider payloads.
Audit logs	Log login, role changes, form edits, submissions, approvals, provider checks, billing changes, document generation.
Webhooks	Validate signatures, reject replay when possible, make processing idempotent.
Error recovery	Provider timeouts create retryable pending states and tasks, not false failures/clears.
Monitoring	Sentry and structured logs for failed jobs, API failures, billing mismatches, and authorization errors.

14. Milestone Delivery Plan

Each milestone should produce a usable, reviewable output. This reduces scope risk and matches pay-on-delivery expectations.

Milestone	Scope	Output / Acceptance Criteria
M1 - Foundation	Repo setup, database schema, auth, workspace model, roles, protected layout,	Users can register/login, create workspace, access protected app, and see workspace-

	base dashboard shell.	scoped data only.
M2 - Restricted Onboarding	Business registration, workspace status, restricted dashboard, setup progress, locked modules.	New workspace lands in restricted dashboard; client onboarding and paid checks are locked with clear explanation.
M3 - Implementation Wizard	Six-step wizard, autosave, validation, conditional sections, progress tracking.	User can complete all six forms, save/exit/resume, and unlock completion state.
M4 - Generated Program Outputs	Risk assessment summary, policy/procedure DOCX/PDF generation, version storage.	Completion creates versioned outputs linked to structured form data. Downloads work securely.
M5 - Manage My Program Base	Program forms update, training record, program review/update record, document versions.	Users can maintain program data and regenerate controlled outputs.
M6 - Client Onboarding Foundation	Customer files, Initial CDD, customer states, evidence upload, basic risk rating.	Workspace can create customer files and progress low-risk onboarding without live paid APIs.
M7 - Escalation and Review Workflows	ECDD, BO, SOF/SOW, escalation, unusual activity review, reviewer decisions.	High-risk path can be reviewed, approved/held/declined, and fully audited.
M8 - Provider API Layer	Provider adapters, check requests/results, webhook gateway, retries, normalized results.	Identity/KYB/AML checks can be triggered and traced from UI to result and raw payload.
M9 - Billing and Usage	Stripe billing, payment method, usage events, invoice/usage dashboard.	Paid checks generate billable events and are blocked when billing is inactive.
M10 - Hardening and Launch	Security review, audit completeness, QA, deployment, monitoring, documentation.	Production-ready MVP with documented handover, tested workflows, and operational logs.

15. Suggested Timeline

The timeline depends heavily on final provider choices, UI polish, final legal templates, and whether the first MVP uses live APIs or mocked/sandbox adapters.

Phase	Scope	Estimated Duration
Phase 1 - Core MVP	Foundation, workspace setup, restricted dashboard, implementation wizard, generated documents, Manage My Program base.	4-6 weeks
Phase 2 - Operational MVP	Customer files, onboarding workflows, ECDD/BO/SOF, escalation, evidence, basic audit trail.	5-7 weeks
Phase 3 - Integrations and Billing	Identity/KYB/AML provider adapters, webhooks, usage events, Stripe billing, retries/reconciliation.	4-6 weeks
Phase 4 - Hardening	Security, permissions, audit verification, QA, deployment, monitoring, handover.	2-4 weeks

Practical advice: start with Phase 1 as the first commercial milestone. Do not commit to all live provider integrations before the providers, API docs, sandbox access, and billing rules are confirmed.

16. Testing and QA Plan

Test Area	Coverage
Unit tests	Permission helpers, state transitions, billing event rules, provider normalizers, document variable mapping.
Integration tests	Registration -> workspace -> setup wizard -> generated outputs; customer onboarding -> checks -> escalation.
RBAC tests	Every role tested against allowed/blocked actions, server-side.
Provider tests	Mock providers, webhook signature validation, duplicate webhook/idempotency, retry behavior.
Billing tests	Stripe test mode subscriptions, payment failure, usage events, invoice

	states.
Security tests	Cross-tenant access attempts, signed URL expiry, restricted record access, audit log immutability checks.
UAT scripts	Client-facing workflows: low-risk individual, low-risk entity, high-risk escalated path, billing inactive path.

17. Open Decisions to Confirm Before Final Quote

1. Which identity, KYB, AML screening, and billing providers are confirmed for MVP?
2. Should billing be required before setup, or only before paid checks?
3. Are AML form questions, risk scoring rules, and document templates final and approved by the client/compliance expert?
4. Should generated documents be editable in the portal, or regenerated only from structured form updates?
5. Which roles are mandatory in MVP, and which can be bundled to reduce cost?
6. Are there final UI designs, or should the build use the provided wireframes and a clean SaaS design system?
7. Will the first milestone use live provider APIs, sandbox APIs, or mocked adapters?
8. What export formats are required: DOCX, PDF, CSV, audit package, or all?
9. How long should evidence, provider raw payloads, and generated documents be retained?

18. Developer Positioning for Meeting

Recommended short explanation to use in the client meeting:

I reviewed the documents and I would treat this as a staged SaaS platform, not a simple form app. The strongest first milestone is the workspace foundation, restricted dashboard, and implementation wizard, because that proves the core product flow and unlocks the generated AML program outputs. After that, we can add client onboarding, paid checks, billing usage, and deeper audit workflows step by step.

My recommendation is to keep the first MVP focused, build the architecture correctly from day one, and avoid hard-coding provider-specific logic so the platform can replace identity/KYB/AML providers later without rewriting the portal.

19. Final Build Strategy

- Lead with a staged milestone plan, not one large undefined fixed scope.
- Start with the core workspace + setup wizard before heavy live integrations.
- Build RBAC, audit, and workspace isolation early because they are expensive to retrofit later.
- Use mocked/sandbox provider adapters first if provider contracts are not final.
- Keep legal/compliance content as client-owned input; implement it as structured fields, templates, and rules.
- Use clean, simple, enterprise UI for MVP. Avoid over-investing in custom design before workflows are validated.
- Make every milestone demoable, testable, and payable against working output.